

Gemini Embedding 2

One model for the rite of all

Turn Text, Images, Documents, Audio & Video into Powerful Embeddings

Why Gemini Embedding 2?

- State-of-the-art embeddings
- Multimodal (Text, Image, PDF, Audio, Video)
- Up to 8,192 text tokens
- High quality, smaller cost
- Built by Google

Freddy Domínguez

- Data Scientist
- Content Creator
- Tech Writer
- AI Enthusiast

> "Embeddings turn meaning into numbers, so machines can understand, compare, and find what really matters."

At a Glance

Model	gemini-embedding-2
Default Dimension	3,072
Max Text Tokens	8,192
Max Images per Request	6
PDF	Up to 6 pages
Audio	Up to 80 seconds
Video	Up to 128 seconds
Output Dimensionality	Customizable (e.g., 10, 256, ...)

1 What are Embeddings?

Embeddings are numerical vectors that capture semantic meaning.

Similar content → vectors are closer in the embedding space.

Different content → vectors are farther apart.



Embeddings enable semantic search, clustering, recommendations, RAG and much more.

2 Get Started: Setup in Colab

```
# 1. Install the SDK
%pip install -q -U "google-genai>=1.73.0"

# 2. Get your API key from Google AI Studio
from google.colab import userdata
GEMINI_API_KEY = userdata.get("GEMINI_API_KEY")

# 3. Initialize the client
from google import genai
from google.genai import types
client = genai.Client(api_key=GEMINI_API_KEY)

# 4. Choose a model
MODEL_ID = "gemini-embedding-2"
```

Get your API key at aistudio.google.com

3 Create Embeddings

T A. Single Text Embedding

```
text = ["Hello world"]
result = client.models.embed_content(
        model=MODEL_ID,
        contents=text
)
[embedding] = result.embeddings
print(len(embedding.values))  # 3072
```

Output

```
0.0123,
-0.0312,
0.0021,
...
(3072 dims)
```

T B. Batch Text Embeddings

```
texts = ["Peru Running", "Data Scientist", "Cevihito"]
result = client.models.embed_content(
        model=MODEL_ID,
        contents=texts
)
for embedding in result.embeddings:
    print(len(embedding.values))
```

C C. Multimodal Embeddings

Text + Image + PDF + Audio + Video

D D. Text + Image Together

```
contents = ["Peru Running",
            types.Part.from_bytes(data=image_bytes,
                mime_type="image/png")]
result = client.models.embed_content(model=MODEL_ID,
        contents=contents)
```

E, Content Objects (Recommended for clarity)

```
content_1 = types.Content(parts=[types.Part.from_text("Peru Running")])
content_2 = types.Content(parts=[types.Part.from_bytes(
        data=image_bytes, mime_type="image/png")])
result = client.models.embed_content(
      model=MODEL_ID, contents=[content_1, content_2])
```

Supports up to 8,192 text tokens, 6 images, 6-page PDF, 80s audio, and 128s video per request.

4 Reduce Embedding Size (Dimensionality)

Reduce the output dimensions to save storage and improve retrieval speed.

```
result = client.models.embed_content(
        model=MODEL_ID,
        contents=["Hello world"],
        config=types.EmbedContentConfig(
            output_dimensionality=10  # from 3072 → 10
        )
)
[embedding] = result.embeddings
print(len(embedding.values))  # 10
```

Smaller vectors = lower cost, faster storage, efficient similarity search.

5 Similarity & Semantic Search

Cosine Similarity (commonly used)

```
from numpy import dot
from numpy.linalg import norm

def cosine_similarity(a, b):
    return dot(a, b) / (norm(a) * norm(b))
```

Example

Query embedding

Item embedding

0.82 (High Similarity)

Use cases

- Find similar documents
- Clustering
- Semantic search
- Recommendations

6 Build a Simple FAQ Semantic Search

1. Prepare your FAQ dataset

- Each question/answer pair
- Embed questions (or answers)
- Store embeddings + metadata

2. Search

```
query_emb = embed(query_text)
scores = []
for item in faq_items:
    score = cosine_similarity(
        query_emb, item["embedding"])
    scores.append((score, item))
results = sorted(scores, key=lambda x:x[0],
          reverse=True)[:top_k]
```

3. Return top matches

Q1	0.93
Q2	0.86
Q3	0.81

...

Return Answer + Source

This is the foundation of RAG systems and modern AI search.

7 Best Practices

- Choose the right input type (text, image, pdf, audio, video).
- Use batch requests for better performance.
- Normalize & store embeddings as float32.
- Use cosine similarity for semantic search.
- Experiment with output_dimensionality to balance quality vs. efficiency.
- Cache embeddings to reduce API calls and cost.
- Keep your data clean and chunk long documents when needed.

8 Practical Use Cases

- **Semantic Search** (Search by meaning, not keywords)
- **RAG (Retrieval-Augmented Generation)** (Find relevant context)
- **Recommendation Systems** (Products, content, users)
- **Image Search** (Find similar images)
- **Document Clustering** (Group similar documents)
- **Multimodal Search** (Combine text, image, audio, video)
- **Classification** (Detect type/category)

9 Example End-to-End Flow

Input (Text / Image / PDF / Audio / Video) → Gemini Embedding 2 (Embeddings) → Vector DB (Store) → Similarity Search → Results / Answer

Transform content → store vectors → search by meaning → get answers!

10 Sample Code Snippet (Python)

```
from google import genai
from google.genai import types
client = genai.Client(api_key=GEMINI_API_KEY)
# Embed text
result = client.models.embed_content(
    model="gemini-embedding-2",
    contents=["Your text or multimodal content here"],
    config=types.EmbedContentConfig(output_dimensionality=256)
)
[embedding] = result.embeddings
print(len(embedding.values))  # 256
```

Default dimension: 3,072 (High quality) → Custom dimension: 256 (Faster & smaller)

Helpful Resources

- Docs: https://ai.google.dev/docs/embeddings
- Model: gemini-embedding-2
- Google AI Studio: https://aistudio.google.com/
- SDK: google-genai (Python)
- Article: https://romellfudi.medium.com/gemini-embedding-2-one-model-for-the-rite-of-all-436583c553ba

Start building smarter AI applications today with Gemini Embedding 2!